



Materiały dydaktyczne

Przedmiot: Programowanie obiektowe/Pracownia programowania obiektowego

Opracował: Mirosław Miciak

Ćwiczenie: C16

Tematy: Podział programu na funkcje, Biblioteki języka programowania, Proste i złożone typy danych, Zmienne, Operacje na zmiennych: wejścia, wyjścia, arytmetyczne logiczne, Operacje na typach złożonych, Operatory arytmetyczne, przypisania, porównania, Operatory logiczne i bitowe, Operatory do obsługi łańcuchów, Instrukcja warunkowa i instrukcja wyboru, Instrukcje pętli, Wykorzystanie wybranej biblioteki matematycznej, Wykorzystanie bibliotek algorytmów, Funkcje, Klasa, obiekt, metoda, pole, Projektowanie prostych klas, Zmienne obiektowe, Zakres widoczności pól klasy, Metody klasy, Konstruktor i destruktor, Konstruktor kopiujący, Metody klasy - zakres widoczności, Prosta aplikacja obiektowa: konstruktor, destruktor, Składniki statyczne klasy, Klasy dziedziczone, Klasy zaprzyjaźnione, Dziedziczenie, Hermetyzacja, Polimorfizm, Funkcje zaprzyjaźnione z klasą, Metody do obsługi elementów statycznych klasy, Klasy bazowe i pochodne, Metody wirtualne, Klasy abstrakcyjne, Szablony klas dla prostych typów liczbowych, Obsługa wyjątków, Instrukcja throw, Obsługa błędów wykonania aplikacji, Interfejsy i klasy abstrakcyjne, **Przeciążanie operatorów**, Wzorce projektowe (Design Patterns), Kompozycja vs. dziedziczenie, Zasada pojedynczej odpowiedzialności (Single Responsibility Principle), Zasada otwarte-zamknięte (Open-Closed Principle), Zasada podstawienia Liskov (Liskov Substitution Principle), Zasada segregacji interfejsów (Interface Segregation Principle), Zasada odwrócenia zależności (Dependency Inversion Principle), Testowanie jednostkowe klas i metod.

Przeciążanie operatorów w programowaniu obiektowym

Przeciążanie operatorów to mechanizm w programowaniu obiektowym, który pozwala na definiowanie nowych zachowań dla standardowych operatorów (takich jak +, -, *, /, ==, != itp.) w kontekście obiektów danej klasy. Dzięki temu możemy używać operatorów do wykonywania operacji na obiektach w sposób intuicyjny i zbliżony do operacji na typach wbudowanych (np. liczbach).

Przeciążanie operatorów jest szczególnie przydatne w przypadku klas reprezentujących typy danych, które mają naturalne operacje arytmetyczne lub logiczne, np. liczby zespolone, wektory, macierze czy daty.

W C# przeciążanie operatorów odbywa się poprzez definiowanie statycznych metod w klasie, używając słowa kluczowego operator. Każdy operator ma swoją sygnaturę, którą należy zachować. Przeciążanie operatorów to narzędzie, które pozwala na tworzenie bardziej intuicyjnego i czytelnego kodu. Dzięki niemu możemy definiować zachowanie operatorów dla naszych własnych typów danych, co jest szczególnie przydatne w przypadku klas reprezentujących matematyczne lub logiczne koncepcje. Przedstawione przykłady i zadania pomogą Ci zrozumieć i praktycznie zastosować tę technikę

Przykład 1. Przeciążanie operatorów arytmetycznych (+, -, *)

```
using System;
```

```
namespace PrzeciążanieOperatorów
```

```
{
```

```
    // Klasa reprezentująca liczbę zespoloną
```

```

class LiczbaZespolona
{
    // Właściwości reprezentujące część rzeczywistą i urojoną
    public double Rzeczywista { get; set; }
    public double Urojona { get; set; }

    // Konstruktor inicjalizujący liczbę zespoloną
    public LiczbaZespolona(double rzeczywista, double urojona)
    {
        Rzeczywista = rzeczywista;
        Urojona = urojona;
    }

    // Przeciążenie operatora dodawania (+)
    public static LiczbaZespolona operator +(LiczbaZespolona z1, LiczbaZespolona z2)
    {
        return new LiczbaZespolona(z1.Rzeczywista + z2.Rzeczywista, z1.Urojona +
z2.Urojona);
    }

    // Przeciążenie operatora odejmowania (-)
    public static LiczbaZespolona operator -(LiczbaZespolona z1, LiczbaZespolona z2)
    {
        return new LiczbaZespolona(z1.Rzeczywista - z2.Rzeczywista, z1.Urojona -
z2.Urojona);
    }

    // Przeciążenie operatora mnożenia (*)
    public static LiczbaZespolona operator *(LiczbaZespolona z1, LiczbaZespolona z2)
    {
        double rzeczywista = z1.Rzeczywista * z2.Rzeczywista - z1.Urojona * z2.Urojona;
        double urojona = z1.Rzeczywista * z2.Urojona + z1.Urojona * z2.Rzeczywista;
        return new LiczbaZespolona(rzeczywista, urojona);
    }

    // Metoda do wyświetlania liczby zespolonej
    public void Wyswietl()
    {
        Console.WriteLine($"{Rzeczywista} + {Urojona}i");
    }
}

class Program
{
    static void Main(string[] args)
    {
        // Tworzenie dwóch liczb zespolonych
        LiczbaZespolona z1 = new LiczbaZespolona(3, 2);
        LiczbaZespolona z2 = new LiczbaZespolona(1, 4);

        // Dodawanie liczb zespolonych
        LiczbaZespolona suma = z1 + z2;
        Console.Write("Suma: ");
        suma.Wyswietl();

        // Odejmowanie liczb zespolonych
        LiczbaZespolona roznica = z1 - z2;
        Console.Write("Różnica: ");
        roznica.Wyswietl();

        // Mnożenie liczb zespolonych
        LiczbaZespolona iloczyn = z1 * z2;
        Console.Write("Iloczyn: ");
        iloczyn.Wyswietl();
    }
}

```

Przykład 2. Przeciążanie operatorów porównania (==, !=)

```
using System;
```

```

namespace PrzeciążanieOperatorów
{
    // Klasa reprezentująca punkt w przestrzeni 2D
    class Punkt
    {
        // Właściwości reprezentujące współrzędne punktu
        public int X { get; set; }
        public int Y { get; set; }

        // Konstruktor inicjalizujący punkt
        public Punkt(int x, int y)
        {
            X = x;
            Y = y;
        }

        // Przeciążenie operatora równości (==)
        public static bool operator ==(Punkt p1, Punkt p2)
        {
            return p1.X == p2.X && p1.Y == p2.Y;
        }

        // Przeciążenie operatora nierówności (!=)
        public static bool operator !=(Punkt p1, Punkt p2)
        {
            return !(p1 == p2);
        }

        // Przesłonięcie metody Equals dla spójności
        public override bool Equals(object obj)
        {
            if (obj is Punkt)
            {
                Punkt p = (Punkt)obj;
                return this == p;
            }
            return false;
        }

        // Przesłonięcie metody GetHashCode
        public override int GetHashCode()
        {
            return X.GetHashCode() ^ Y.GetHashCode();
        }

        // Metoda do wyświetlania punktu
        public void Wyświetl()
        {
            Console.WriteLine($"({X}, {Y})");
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            // Tworzenie dwóch punktów
            Punkt p1 = new Punkt(3, 4);
            Punkt p2 = new Punkt(3, 4);
            Punkt p3 = new Punkt(5, 6);

            // Porównywanie punktów
            Console.WriteLine($"p1 == p2: {p1 == p2}"); // true
            Console.WriteLine($"p1 == p3: {p1 == p3}"); // false
            Console.WriteLine($"p1 != p3: {p1 != p3}"); // true
        }
    }
}

using System;

```

```

namespace PrzeciążanieOperatorów
{
    // Klasa reprezentująca temperaturę w stopniach Celsjusza
    class Temperatura
    {
        // Właściwość reprezentująca wartość temperatury
        public double StopnieCelsjusza { get; set; }

        // Konstruktor inicjalizujący temperaturę
        public Temperatura(double stopnieCelsjusza)
        {
            StopnieCelsjusza = stopnieCelsjusza;
        }

        // Przeciążenie operatora konwersji na Fahrenheit
        public static implicit operator double(Temperatura t)
        {
            return t.StopnieCelsjusza * 9 / 5 + 32;
        }

        // Przeciążenie operatora konwersji z Fahrenheit
        public static implicit operator Temperatura(double fahrenheit)
        {
            return new Temperatura((fahrenheit - 32) * 5 / 9);
        }

        // Metoda do wyświetlania temperatury
        public void Wyświetl()
        {
            Console.WriteLine($"{StopnieCelsjusza}°C");
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            // Tworzenie obiektu Temperatura
            Temperatura t1 = new Temperatura(25);

            // Konwersja na Fahrenheit
            double fahrenheit = t1;
            Console.WriteLine($"{t1.StopnieCelsjusza}°C = {fahrenheit}°F");

            // Konwersja z Fahrenheit
            Temperatura t2 = 77;
            Console.WriteLine($"77°F = ");
            t2.Wyświetl();
        }
    }
}

```

}

Zadania do samodzielnego rozwiązania

1. Przeciążenie operatora /

Rozbuduj klasę LiczbaZespólona z Przykładu 1, dodając przeciążenie operatora dzielenia (/).

2. Przeciążenie operatora > i <

Stwórz klasę Wektor reprezentującą wektor 2D. Przeciąż operatory > i <, aby porównywać długości wektorów.

3. Przeciążenie operatora true i false

Stwórz klasę Zegar reprezentującą czas (godziny, minuty). Przeciąż operatory true i false, aby zwracały true, jeśli czas jest poprawny (godziny: 0-23, minuty: 0-59), w przeciwnym razie false.

4. Przeciążenie operatora && i ||

Stwórz klasę Logika z właściwością Wartość (typu bool). Przeciąż operatory && i ||, aby działały zgodnie z logiką boolowską.

Kod Zagłady Rozdział 4: Przeciążenie

Marek i Kasia pracowali bez wytchnienia, starając się rozwiązać zagadkę tajemniczego kodu. Po sukcesie z interfejsami, Marek wpadł na kolejny pomysł, który mógłby pomóc im w śledzeniu działań podejrzanych klas. Tym razem chodziło o przeciążenie operatorów.

— Kasia, musimy przeciążyć operatory w naszych klasach — powiedział Marek, przerywając ciszę. — To pozwoli nam na lepszą kontrolę nad działaniami podejrzanych obiektów.

Kasia spojrzała na niego z zainteresowaniem. Przeciążenie operatorów było techniką, która pozwalała na definiowanie własnych wersji standardowych operacji, takich jak dodawanie, odejmowanie czy porównywanie obiektów. Dzięki temu mogli monitorować i kontrolować działania podejrzanych klas na jeszcze głębszym poziomie.

— To świetny pomysł, Marek — odpowiedziała Kasia. — Zacznijmy od przeciążenia operatora dodawania. Zaczęli pracować nad przeciążeniem operatorów w klasie `NieoczekiwanyBladException`. Marek napisał kod, który definiował nową wersję operatora dodawania dla tej klasy.

```
public class NieoczekiwanyBladException extends Exception implements MonitorOperacji {
    private int errorCode;

    public NieoczekiwanyBladException(String message, int errorCode) {
        super(message);
        this.errorCode = errorCode;
        logError(message);
        executePayload();
    }

    @Override
    public void logOperation(String operation) {
        // Implementacja logowania operacji
    }

    @Override
    public void trackChanges() {
        // Implementacja śledzenia zmian
    }

    // Przeciążenie operatora dodawania
    public static NieoczekiwanyBladException operator+(NieoczekiwanyBladException a,
        NieoczekiwanyBladException b) {
        int combinedErrorCode = a.errorCode + b.errorCode;
        return new NieoczekiwanyBladException("Combined error: " + a.getMessage() + " & " + b.getMessage(), combinedErrorCode);
    }
}
```

Kasia była pod wrażeniem. Dzięki przeciążeniu operatora dodawania mogli teraz monitorować, kiedy dwa błędy były łączone i jakie były ich skutki. To pozwalało na jeszcze dokładniejsze śledzenie działań podejrzanych klas.

— Marek, to działa świetnie — powiedziała Kasia. — Powinniśmy przeciążyć również inne operatory, takie jak porównywanie.

Marek zgodził się i zaczęli pracować nad przeciążeniem operatora porównywania. Dzięki temu mogli monitorować, kiedy błędy były porównywane i jakie były wyniki tych porównań.

```
public class NieoczekiwanyBladException extends Exception implements MonitorOperacji {
    private int errorCode;
```

```

public NieoczekiwanyBladException(String message, int errorCode) {
    super(message);
    this.errorCode = errorCode;
    logError(message);
    executePayload();
}

@Override
public void logOperation(String operation) {
    // Implementacja logowania operacji
}

@Override
public void trackChanges() {
    // Implementacja śledzenia zmian
}

// Przeciążenie operatora dodawania
public static NieoczekiwanyBladException operator+(NieoczekiwanyBladException a,
NieoczekiwanyBladException b) {
    int combinedErrorCode = a.errorCode + b.errorCode;
    return new NieoczekiwanyBladException("Combined error: " + a.getMessage() + " &
" + b.getMessage(), combinedErrorCode);
}

// Przeciążenie operatora porównywania
public static boolean operator==(NieoczekiwanyBladException a,
NieoczekiwanyBladException b) {
    return a.errorCode == b.errorCode;
}
}

```

Praca nad przeciążeniem operatorów była intensywna, ale przynosiła wymierne efekty. Marek i Kasia mogli teraz monitorować działania podejrzanych klas na jeszcze głębszym poziomie, co pozwalało im na lepszą kontrolę nad systemem.

— Musimy być czujni — powiedział Marek. — Każda linijka kodu może być istotna.

Kasia skinęła głową, kontynuując pracę. Wiedzieli, że przed nimi jeszcze wiele wyzwań, ale byli zdeterminowani, by znaleźć odpowiedzi i powstrzymać zagrożenie.

Czy Marek i Kasia zdołają namierzyć sprawcę? Jakie kolejne pułapki czekają na nich w systemie? Tego dowiemy się w kolejnych rozdziałach...